THS-II · EMS
Reinforcement Learning Pipeline

Enhanced Sprint Guide — v2.0

Toyota THS-II · PPO Agent · GPS Route Planning · Custom Gymnasium · ONNX / Raspberry Pi HIL

Vehicle	Toyota THS-II — Power-Split Hybrid (Prius ZVW30)
RL Algorithm	PPO — Proximal Policy Optimisation (Stable-Baselines3)
Physics Engine	modeling1.py v4 — 2ZR-FXE · PSD · MG1/MG2 · NiMH
Gymnasium	Custom env wrapping THSIIController.step()
Drive Cycles	WLTC Class 3 · FTP-75/UDDS · US06
GPS/Routing	OpenRouteService API · Elevation · Traffic · Segment Classification
Drive Modes	ECO · NORMAL · PWR · EV · SPORT (5 modes — THS-II aligned)
Timeline	3-Day Sprint (Core) + Day 4 GPS/Route + Day 5 CARLA Bonus + Day 6 CAN HIL
HIL Platform	Raspberry Pi 4 — 4 GB RAM [Optional Day 6]
Dashboard	Streamlit web app — live SOC, fuel, GPS telemetry

CONFIDENTIAL — All instructions are implementation-ready starting points.

Oussama Chouaibi · EABA | INSAT — Industrial IT & Automation | 2025/2026

Table of Contents

1 · Introduction & Problem Statement

The Toyota Hybrid System II (THS-II), deployed across the Prius and Lexus families, uses a planetary power-split device (PSD) that allows the internal combustion engine (ICE), Motor-Generator 1 (MG1), and Motor-Generator 2 (MG2) to operate simultaneously or independently. This architecture makes energy management a complex, multi-dimensional optimisation problem that is poorly served by static rule tables.

This project replaces conventional hand-crafted EMS policies with a Reinforcement Learning agent trained entirely on the high-fidelity modeling1.py physics engine — a production-grade THS-II v4 simulator with real BSFC maps, planetary gear dynamics, MG1/MG2 efficiency lookup tables, a NiMH thermal model, DC-DC converter model, and blended braking logic. The v2.0 enhanced pipeline adds GPS-based route segmentation, road topology analysis, elevation and traffic integration, and a dedicated Day 4 for predictive EMS.

1.1 Project Objectives

ID	Objective
OBJ-01	Wrap modeling1.py physics inside a standards-compliant Gymnasium environment
OBJ-02	Train a PPO agent to select optimal driving modes per road segment type
OBJ-03	Minimise total fuel consumption over standardised drive cycles (WLTC, FTP-75, US06)
OBJ-04	Maintain battery SOC within ±5 % of the 60 % charge-sustaining reference
OBJ-05	Integrate GPS route data — elevation, segment type, traffic — as predictive state features
OBJ-06	Export the trained policy to ONNX; validate inference on Raspberry Pi 4
OBJ-07	Expose results through an interactive Streamlit web dashboard with GPS map
OBJ-08	[Optional D5] Integrate RL mode output with live CARLA traffic co-simulation
OBJ-09	[Optional D6] Validate ONNX policy in CAN Bus Hardware-in-the-Loop on RPi 4

1.2 Scope & Constraints

Parameter	Value
Physics Engine	modeling1.py v4 — 2ZR-FXE, PSD, MG1/MG2, NiMH — DO NOT MODIFY
Drive Modes	ECO · NORMAL · PWR · EV · SPORT (5 discrete actions — THS-II DriveMode enum)
Drive Cycles	WLTC Class 3 · FTP-75/UDDS · US06 (.csv files)
GPS / Route	OpenRouteService API or local OSRM — elevation + traffic labels per segment
Timeline	3 days core + 1 GPS day + 1 optional CARLA + 1 optional CAN HIL
HIL Platform	Raspberry Pi 4 (4 GB RAM) + MCP2515 CAN HAT [Optional D6]
Budget	~€60 hardware (RPi 4 + CAN HAT)

2 · State of the Art

2.1 Rule-Based EMS

Early EMS implementations use deterministic switching rules derived from expert knowledge (charge-depleting / charge-sustaining strategies, thermostat control). They are computationally cheap but cannot account for route topology or traffic context and require manual calibration per vehicle variant.

2.2 Optimal Control Methods

Dynamic Programming (DP) yields globally optimal policies over a known drive cycle but requires full future trip knowledge — making it acausal. Pontryagin's Minimum Principle and ECMS reduce cost but rely on heuristic co-state estimation. MPC adds a rolling prediction horizon at the cost of solving a quadratic programme every timestep.

2.3 Learning-Based EMS — This Work

PPO-based agents have shown 12–15 % fuel reductions on standard cycles with SOC deviations below 3 %. The distinguishing contribution here is threefold: (1) the high-fidelity modeling1.py engine with real lookup tables — not a toy model; (2) GPS-driven predictive state enrichment that gives the agent road topology context before each segment; (3) a low-cost HIL pipeline that exports to ONNX for real-time inference on a Raspberry Pi 4, replicable without proprietary hardware.

2.4 GPS & Route Planning Approaches

Route-aware EMS requires segment-level data: road type, slope, speed limits, and traffic density. Map-matching libraries (OSRM, Valhalla) and elevation APIs (Open-Elevation, SRTM) provide this data. Route segmentation divides a trip into homogeneous chunks (urban / rural / highway) using speed-limit topology and OSM road class tags, enabling the RL agent to receive a predictive lookahead state.

2.5 Simulation Platforms

CARLA 0.9.x provides realistic traffic, road geometry, and sensor fusion for extended validation. For standard drive-cycle training, the standalone Gymnasium environment is sufficient and orders of magnitude faster than CARLA-in-the-loop training.

2.6 THS-II vs THS-V

Feature	THS-II	THS-V
Generation	THS-II (Prius ZVW30)	THS-V (Prius 60-series)
Efficiency	Good	Better (+~8 %)
Complexity	Medium	Higher
Research maturity	Mature / well-documented	Newer — fewer open datasets
This project	✓ Used (modeling1.py v4)	Future work (FW-04)

3 · Proposed Architecture & Solution

The system is structured around three mandatory phases (SIL training + GPS integration + evaluation) and two optional extension phases (CARLA co-simulation on Day 5, CAN HIL on Day 6).

3.1 Phase Overview

Phase	Description	Key Action	Output
Phase 1 — Training (PC)	Gymnasium env wrapping modeling1.py	PPO agent learns mode-switching	SB3 checkpoint saved
Phase 2 — GPS Route (PC)	ORS/OSRM route data → segment features	Predictive state enrichment	GPS-aware episodes
Phase 3 — Evaluation (PC)	SIL validation on all 3 drive cycles	ONNX export	Streamlit dashboard
Phase 4 — CARLA [Opt D5]	RL mode output replaces rule selector	CarlaTHSIISimulation subclass	Live traffic validation
Phase 5 — CAN HIL [Opt D6]	ONNX policy runs on RPi 4	States/modes via CAN Bus 2.0B	500 kbit/s embedded validation

3.2 Driving Modes — Full 5-Mode THS-II Alignment

Drive Mode Expansion

This v2.0 guide expands the action space from 4 to 5 modes to match the complete THS-II DriveMode enum including SPORT. SPORT differs from PWR in that it allows peak MG2 torque boost with reduced SOC protection — used on steep grades or overtaking scenarios. The AUTOMATIC mode is reserved for the physics engine internal fallback and is NOT exposed as an RL action.

Action	Behaviour	Target Segment	GPS Trigger Condition
0 — ECO	Conservative regen, reduced EV threshold, low ICE load	Urban stop-go	≤ 15 km/h segments
1 — NORMAL	Balanced ICE/motor torque split, standard regen	Mixed roads	15 – 80 km/h
2 — PWR	Higher ICE output, aggressive MG2 boost, relaxed SOC target	Highway / grade	≥ 80 km/h
3 — EV	Pure electric below 72 km/h, SOC ≥ 45 % required	Low-speed urban	Queue / crawl traffic
4 — SPORT	Peak MG2 torque burst, maximum ICE power, SOC protection relaxed	Steep grade / overtake	High demand transient

3.3 Training vs. GPS-Predictive vs. HIL Flow

Aspect	Training Phase	GPS-Predictive Phase	HIL Validation [Opt]
Environment	Gymnasium wrapping modeling1.py	Same Gym + GPS feature injection	Same Gym sim on PC + RPi CAN
Agent location	PC — SB3 training loop	PC — GPS-enriched state	PC sends states via CAN frame
Inference	PC — policy.predict()	PC — policy.predict() + GPS lookahead	Raspberry Pi 4 — ONNX Runtime
Timestep	Variable (fast rollout)	Variable + route pre-computed	Fixed 100 ms CAN frame cycle
Purpose	Learn optimal policy	Route-aware generalisation	Validate embedded deployment

4 · System Components & Technologies

4.1 The modeling1.py Physics Engine

modeling1.py (THS-II v4) is the powertrain physics core. Its THSIIController class exposes a .step(throttle, brake, speed, grade, dt) method returning a full telemetry dictionary every timestep. The Gymnasium environment wraps this controller directly — no simplified approximations.

Parameter	Value
ICE Model	2ZR-FXE Atkinson 73 kW — BSFC map (RPM × Torque), friction FMEP, coolant thermal
Power Split Device	Planetary K=2.6, dual-mesh efficiency 0.980
MG1 (Sun gear)	42 kW peak, 10 000 rpm limit, 7×7 efficiency table
MG2 (Ring/Output)	60 kW peak, 207 Nm, reduction 2.636×3.267, 7×7 efficiency table
HV Battery	NiMH 201.6 V / 6.5 Ah, RC thermal model, OCV-SOC curve, SOC 40–80 %
DC-DC Converter	500 V HV bus, boost η=0.972, buck η=0.968
Blended Braking	Hydraulic fade-in above 0.3 g decel, regen priority
EV Mode Cap	72 km/h (20 m/s), SOC ≥ 45 % required
ICE Anti-Hunt	Dual threshold P_on=4 kW / P_off=1.5 kW + 10 s minimum warmup hold
SPORT Trigger	MG2 torque cap raised to 207 Nm sustained, SOC target relaxed to 55 %

4.2 Custom Gymnasium Environment

The environment wraps THSIIController. The state space exposes 8 variables (5 powertrain + 3 GPS-derived); the action space maps to the 5 DriveMode values. The reward penalises fuel consumption and SOC deviation while rewarding regenerative energy capture.

State Vector (8 dimensions — v2.0 GPS-Enriched):

Variable	Unit	Source	Description
velocity	m/s	speed arg / drive cycle	Current vehicle speed (normalised ÷ 30)
SOC	[0,1]	state.soc_pct / 100	Battery state-of-charge
road_grade	rad	GPS elevation gradient / grade arg	Segment slope (normalised ÷ 0.3)
segment_type	{0,1,2}	GPS road class / speed-derived	0=urban, 1=rural, 2=highway (÷ 2)
acceleration	m/s²	Δspeed/dt	Instantaneous acceleration (÷ 5)
gps_lookahead_grade	rad	Next-segment elevation gradient	Preview of upcoming slope (÷ 0.3)

Variable	Unit	Source	Description
gps_traffic_density	[0,1]	ORS / real-time traffic API	Traffic congestion level (0=free, 1=jam)
gps_dist_to_next_seg	m	Route segment boundary distance	Distance to segment type change (÷ 1000)

Reward Function:

Term	Expression	Purpose
Fuel penalty	- out["fuel_rate_gs"]	Minimise instantaneous fuel rate
SOC penalty	- 10.0 × (soc - 0.60)²	Keep SOC at 60 % reference
Regen bonus	+ 0.5 × max(0, - out["p_batt_kw"])	Reward regenerative braking capture
Grade anticipation bonus	+ 0.3 × (EV on flat) — 0.2 × (EV on grade > 5 %)	Penalise EV on steep uphill (GPS-aware)
SPORT overuse penalty	- 2.0 × SPORT_steps / total_steps	Discourage excessive SPORT selection

4.3 PPO Hyperparameters

Parameter	Value
Algorithm	PPO — Proximal Policy Optimisation
Policy Network	MlpPolicy — 2×64 hidden layers, tanh activation
Learning Rate	3×10^{-4} with linear decay
n_steps	2048
Batch Size	64
n_epochs	10
Clip Range	0.2
GAE Lambda	0.95
Discount γ	0.99
Total Timesteps	500 000 (SIL) / 800 000 (GPS-enriched episodes recommended)
Train Time	~10 min on laptop CPU (~6 min with GPU)
Observation Space	Box(8,) — 5 powertrain + 3 GPS features
Action Space	Discrete(5) — ECO / NORMAL / PWR / EV / SPORT

4.4 Technology Stack

Parameter	Value
Python 3.11	Core language
modeling1.py (v4)	High-fidelity THS-II powertrain physics engine — DO NOT MODIFY
Gymnasium 0.29	RL environment API wrapper
Stable-Baselines3 2.x	PPO training framework
PyTorch 2.x	Neural network backend
OpenRouteService API	GPS route data, elevation profile, segment classification
OSRM (local)	Optional local routing engine for offline GPS route queries
Open-Elevation API	SRTM-based elevation data along GPS routes
ONNX Runtime 1.17	Embedded inference on RPi 4 [Optional Day 6]
python-can 4.x	CAN Bus communication [Optional Day 6]
NumPy / Pandas	Numerical computation & drive cycle data I/O
Matplotlib / Plotly	Result visualisation & publication figures
Folium / Leaflet	GPS route map rendering in Streamlit dashboard
Streamlit 1.x	Interactive web dashboard
CARLA 0.9.x	Live traffic co-simulation [Optional Day 5]

5 · Reinforcement Learning Design

5.1 MDP Formulation

Parameter	Value
State S	8-dimensional vector: [velocity, SOC, road_grade, segment_type, acceleration, gps_lookahead_grade, gps_traffic_density, gps_dist_to_next_seg]
Action A	Discrete {0:ECO, 1:NORMAL, 2:PWR, 3:EV, 4:SPORT} — mirrors full DriveMode enum
Reward R	-fuel_rate_gs - λ·(SOC-0.60)² + γ_regen·regen_energy + gps_anticipation_bonus - sport_overuse_penalty
Transition T	THSIIController.step() — full powertrain dynamics, no approximation
Episode	One full drive cycle (WLTC: 1800 s · FTP-75: ~2474 s · US06: 596 s) or one GPS route
Discount γ	0.99 — long-horizon fuel and SOC shaping

5.2 Drive Cycle & GPS Segmentation Strategy

Condition	Segment	Expected Mode	GPS Context
v < 15 km/h	Urban (segment_type = 0)	ECO or EV preferred	High traffic density expected
15 ≤ v < 80 km/h	Rural (segment_type = 1)	NORMAL preferred	Mixed — GPS grade drives sub-mode
v ≥ 80 km/h	Highway (segment_type = 2)	PWR or NORMAL preferred	SPORT only if GPS grade > 4 %
GPS: grade > 4 %	Uphill (any segment)	PWR or SPORT	Pre-emptive from lookahead
GPS: grade < -2 %	Downhill (any segment)	ECO (max regen)	Lookahead triggers early ECO
GPS: traffic > 0.7	Stop-go urban	EV preferred	Queue anticipation

5.3 Reward Tuning Guidelines

Symptom	Likely Cause	Fix	Action
SOC drops below 45 %	lambda_soc too low	Increase lambda_soc: 10 → 15 → 20	Increase penalty weight
Agent always selects EV mode	Regen bonus too high	Reduce gamma_regen: 0.5 → 0.2	Balance regen reward
Reward flat after 100k steps	Learning rate too high	Decay LR: 3e-4 → 1e-4	Add LR scheduler

Symptom	Likely Cause	Fix	Action
Fuel not improving vs baseline	Action not applied to ems	Verify DriveMode is set in step()	Check selector_mode attr
Episode terminates too early	SOC cutoff too strict	Check the soc < 0.40 condition	Widen SOC floor temporarily
Mode oscillation every few steps	Insufficient hysteresis	Add mode hold counter (≥ 3 steps)	Add hold state variable
SPORT selected too often	SPORT penalty too low	Increase sport_penalty: 2.0 → 4.0	Re-tune penalty term
GPS features ignored	Feature scaling wrong	Verify all GPS obs normalised to [0,1]	Check _obs() normalisation

5.4 Evaluation Metrics

Metric	Definition	Target
Fuel Savings (%)	RL vs rule-based baseline on test cycle	> 5 %
SOC Deviation	RMSE of SOC from 60 % across full episode	< ±5 %
Mode Distribution	Histogram of mode selections per segment type	Qualitative
GPS Anticipation Rate	% of grade events where mode set before segment	> 60 %
Inference Latency (PC)	ONNX sess.run() time on PC	< 2 ms
Inference Latency (RPi)	ONNX sess.run() time on RPi 4 [Optional HIL]	< 10 ms
CAN Reliability	Dropped / corrupt frames [Optional HIL]	< 0.1 %
RAM Footprint	Peak RAM on RPi during HIL [Optional HIL]	< 200 MB
CPU Usage	Average CPU load on RPi [Optional HIL]	< 30 %

6 · GPS Route Planning & Road Topology Integration

This chapter defines the GPS pipeline that enriches the RL state vector with predictive route knowledge. The pipeline is independent of modeling1.py and can be swapped between online (API-based) and offline (OSRM/local) modes without touching the physics engine.

6.1 Architecture Overview

GPS Pipeline Data Flow

- Origin + Destination → ORS/OSRM API → Route GeoJSON (waypoints + road class)
- Route GeoJSON + Open-Elevation API → Elevation profile per waypoint (SRTM 30 m resolution)
- Elevation profile → Gradient computation (grade_rad = atan(Δh / Δd) per segment)
- Road class tags (OSM: motorway / primary / residential) → segment_type labels
- ORS traffic extension or HERE Traffic API → traffic_density per segment
- Pre-trip: full route processed → list of RouteSegment objects saved to route_cache.json
- At runtime: THSEnv reads route_cache.json and injects GPS features each step

6.2 Route Segmentation Strategy

Each route is divided into segments at road class transition points and speed-limit boundaries. The segmentation algorithm uses OSM road class tags as the primary signal, with speed-derived fallback for CSV-only drive cycles.

OSM Road Class	Segment Type	Speed Range	Preferred Mode
motorway / motorway_link	highway (segment_type=2)	≥ 80 km/h	PWR or NORMAL
primary / secondary / trunk	rural (segment_type=1)	15 – 80 km/h	NORMAL
residential / tertiary / unclassified	urban (segment_type=0)	< 15 km/h	ECO or EV
living_street / pedestrian	urban crawl (segment_type=0)	< 5 km/h	EV mandatory
Any + grade > 4 %	uphill override	any	PWR or SPORT
Any + grade < -2 %	downhill override	any	ECO (max regen)

6.3 Elevation & Gradient Computation

Grade is computed from the SRTM-30m elevation profile sampled at each GPS waypoint. A Savitzky-Golay filter (window=11, poly=2) smooths noisy SRTM data before gradient estimation.

Gradient Computation Formula

- grade_rad[i] = atan2(elevation[i+1] - elevation[i], haversine_distance(waypoint[i], waypoint[i+1]))
- Haversine distance: $d = 2R \cdot \arcsin(\sqrt{\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)})$
- Positive grade → uphill → PWR / SPORT anticipation
- Negative grade → downhill → ECO / max regen anticipation

- Lookahead window: 500 m ahead of current vehicle position → gps_lookahead_grade in state

6.4 Traffic & Environmental Data Integration

Traffic density is queried from the ORS directions API (duration_typical vs. duration fields) or from a HERE / TomTom traffic tile overlay. Values are normalised to [0,1] per segment and injected as gps_traffic_density in the observation.

Traffic Density [0,1]	Recommended Mode Bias
Free flow (< 0.3)	NORMAL or PWR — maintain speed, ICE-efficient operating point
Moderate (0.3 – 0.6)	ECO — lower EV threshold, pre-charge for stop-go
Heavy (0.6 – 0.8)	EV priority — frequent stops favour pure-electric short bursts
Jam (> 0.8)	EV mandatory if SOC ≥ 45 %, else ECO with maximum regen on decel

6.5 RouteSegment Data Structure

The GPS pipeline produces a list of RouteSegment objects, serialised to route_cache.json before training. Each object carries all pre-computed features for one road segment.

RouteSegment Fields (route_cache.json schema)

- segment_id: int — sequential index along route
- start_lat, start_lon, end_lat, end_lon: float — GPS coordinates
- length_m: float — haversine segment length in metres
- road_class: str — OSM road class tag (motorway / primary / residential / …)
- segment_type: int — 0=urban, 1=rural, 2=highway
- elevation_start_m, elevation_end_m: float — SRTM elevation
- grade_rad: float — road gradient in radians (positive=uphill)
- speed_limit_kmh: float — OSM maxspeed tag or inferred
- traffic_density: float — [0,1] from ORS or HERE API
- recommended_mode: int — rule-based reference (for baseline comparison)

6.6 GPS Feature Injection into THSEnv

The THSEnv.__init__() method accepts an optional route_cache path. When provided, each step() call queries the pre-computed RouteSegment list by matching current distance_travelled to segment boundaries, and injects GPS features into the 8-dimensional observation. For standard drive-cycle training without GPS, the three GPS dimensions default to zero.

GPS Injection Logic (pseudocode)

- current_seg = find_segment_by_distance(self.distance_m, self.route_segments)
- next_seg = route_segments[current_seg.id + 1] # lookahead
- obs[5] = np.clip(next_seg.grade_rad / 0.3, -1, 1) # gps_lookahead_grade
- obs[6] = current_seg.traffic_density # gps_traffic_density
- obs[7] = (next_seg.start_dist_m - self.distance_m) / 1000.0 # gps_dist_to_next_seg

- self.distance_m += speed_ms * dt # integrate vehicle position

DAY 1 · MON
Environment Setup & Physics Verification
Foundation · All subsequent days depend on this

Day 1 is non-negotiable.

- Every downstream task assumes a verified, fully-functional modeling1.py standalone run and a properly configured Python environment.
- Do not proceed to Day 2 until all checkpoints below are green.
- The Day 4 GPS pipeline requires the directory structure set up here — populate it correctly now.

Part 1A · Python Stack Installation

- Create a dedicated virtual environment (Python 3.11 recommended) and activate it before installing anything.
- Install the RL and simulation stack: stable-baselines3[extra], gymnasium==0.29, torch (CPU build unless you have CUDA), onnxruntime, tensorboard.
- Install the data and visualisation stack: numpy, pandas, matplotlib, plotly, streamlit, folium, streamlit-folium.
- Install the GPS stack: requests, openrouteservice, geopy, scipy (for Savitzky-Golay filter), shapely.
- Install optional packages: python-can (for CAN HIL), carla (only if CARLA is installed on your machine).
- Freeze requirements into requirements.txt immediately after installation so the environment is reproducible.
- Verify the installation by importing each major package in a Python REPL — any ImportError must be resolved before continuing.

Part 1B · Project Directory Structure

Parameter	Value
env/	Gymnasium environment module (ths_env.py + drive cycle CSVs)
env/drive_cycles/	WLTC.csv, FTP75.csv, US06.csv — speed_ms column required
gps/	GPS pipeline scripts: route_fetcher.py, elevation.py, segmenter.py, route_cache.json
gps/cache/	Pre-computed route JSON files for reuse across training runs
training/	PPO training script, baseline comparator, export script
eval/	SIL evaluation, plotting, and metrics scripts
models/	Saved SB3 checkpoints and ONNX model output
hil/	CAN Bus PC-side and RPi controller scripts [Optional Day 6]
carla/	CARLA RL bridge subclass [Optional Day 5]

Parameter	Value
app/	Streamlit dashboard script
tb_logs/	TensorBoard log directory (auto-created by training script)
modeling1.py	THS-II physics engine — place at project root, DO NOT MODIFY

Part 1C · Physics Verification — modeling1.py Standalone Run

- Run modeling1.py with the --standalone flag. Confirm it prints KPI values and exits without errors.
- Run again with --standalone --ftp75. All 2474 timesteps must complete. Note the final total_fuel_g and soc_final values.
- Run with --standalone --plot. Verify the 4×2 subplot PNG is saved to /tmp/ and all subplots are populated.
- Open ths2_kpis.csv. Confirm it has 28 columns and zero NaN values in the SOC or fuel_rate_gs columns.
- Inspect the THSIIController class signature: note the init_drive_mode parameter, the state dataclass fields, and the exact keys returned by .step().
- Note the DriveMode enum values (ECO, NORMAL, PWR, EV, SPORT, AUTOMATIC). Confirm SPORT is present and maps to action 4. AUTOMATIC is reserved for internal use — do NOT expose it as an RL action.
- Document the standalone fuel total for WLTC and FTP-75 — this becomes your physics-engine baseline for Day 3.

Part 1D · Drive Cycle Data Preparation

- Download WLTC Class 3 from the official UNECE source. Convert to CSV: speed_ms column, 1800 rows.
- Download FTP-75/UDDS from the US EPA. Same format: speed_ms column, ~2474 rows.
- Download US06 from the US EPA. Same format: speed_ms column, 596 rows.
- Optionally add a road_grade_rad column (zeros for flat cycle) to each CSV — the GPS pipeline will overwrite this on Day 4.
- Place all three files in env/drive_cycles/ and verify row counts before continuing.

Day 1 — Checkpoints (All Must Pass)

- ✓ python modeling1.py --standalone → prints KPIs, zero errors
- ✓ python modeling1.py --standalone --ftp75 → 2474 steps complete, fuel_g and soc_final printed
- ✓ --plot flag → 4×2 subplot PNG saved, all 8 subplots populated
- ✓ ths2_kpis.csv → 28 columns, no NaN in SOC or fuel_rate_gs
- ✓ WLTC.csv / FTP75.csv / US06.csv present in env/drive_cycles/ with correct row counts
- ✓ All Python imports (SB3, Gymnasium, PyTorch, ONNX, Streamlit, ORS, folium) succeed in REPL
- ✓ Directory structure created with gps/ and gps/cache/ subdirectories present

DAY 2 · TUE
Gymnasium Env + PPO Training
Core RL pipeline · Reward tuning · Baseline comparison

Part 2A · Implementing env/ths_env.py

- __init__: Accept cycle ('WLTC','FTP75','US06'), dt=0.1, and optional route_cache=None parameters. Define observation_space as Box(8,) with low=-1, high=1, dtype=float32 (signed to accommodate negative grades and lookahead values). Define action_space as Discrete(5).
- reset(): Instantiate a new THSIIController(init_drive_mode=DriveMode.ECO) each call. Load drive cycle CSV. If route_cache is not None, load the RouteSegment list from the JSON file. Reset idx=0, distance_m=0.0, self.speed=0.0. Return initial obs and empty info dict.
- step(): Receive integer action (0–4). Map to DriveMode enum (0→ECO, 1→NORMAL, 2→PWR, 3→EV, 4→SPORT). Set ems.state.selector_mode. Compute throttle and brake from speed delta. Call ems.step(). Extract soc, fuel_rate_gs, p_batt_kw. Update distance_m. Compute GPS features if route_cache loaded. Compute reward. Advance idx.
- _obs(): Build the 8-element normalised observation. GPS dims default to 0.0 if route_cache is None — this makes the env backward-compatible for standard drive-cycle training.
- Termination: done=True when idx reaches end of cycle OR soc < 0.40. Truncated=False always.
- Guard against CARLA/pygame at module level with try/except ImportError blocks.

Part 2B · Environment Validation with check_env

- Run check_env(THSEnv('WLTC')) before any training. Resolve every warning — common issues: obs dtype mismatch (must be float32), reward not a float scalar, wrong number of return values from step().
- Run a manual episode with random actions. Verify obs[5:8] are all 0.0 (GPS dims, no route_cache). Plot SOC and fuel_rate_gs.
- Run again with a sample route_cache loaded. Verify obs[5:8] contain non-zero values and change between segments.
- Confirm the done flag at step 1800 for WLTC. Print DriveMode each step — confirm SPORT (action 4) is correctly applied.

Part 2C · Rule-Based Baseline (training/baseline_rule.py)

- Instantiate THSEnv('WLTC') and run with rule: EV if v < 5 km/h AND soc ≥ 0.45; ECO if v < 15 km/h; NORMAL if v < 80 km/h; PWR otherwise. Never select SPORT in the rule-based baseline.
- Record total fuel (sum of fuel_rate_gs × dt) in grams. Record SOC trajectory.
- Save these numbers — PPO must beat this by ≥ 5 % on Day 3.

Part 2D · PPO Training (training/train_ppo.py)

- Instantiate THSEnv('WLTC') for training and a separate instance for evaluation. Never share instances.
- Configure PPO: learning_rate=3e-4, n_steps=2048, batch_size=64, n_epochs=10, gamma=0.99, gae_lambda=0.95, clip_range=0.2, tensorboard_log='./tb_logs/'.

- Add EvalCallback: eval_freq=50_000, best_model_save_path='./models/'.
- Run model.learn(total_timesteps=500_000). Monitor ep_rew_mean in TensorBoard — should rise within first 100k steps.
- If flat after 150k, apply reward tuning from Section 5.3 before adding more timesteps.
- Save final model as models/ths_agent_final. Best checkpoint auto-saved as models/best_model.zip.

Day 2 — Checkpoints (All Must Pass)

- ✓ check_env(THSEnv('WLTC')) → zero warnings or errors
- ✓ Random episode completes 1800 steps, done flag raised cleanly, no crash
- ✓ SOC stays in [0.40, 0.80] across random episode — plot confirms
- ✓ SPORT action (4) correctly sets DriveMode.SPORT — verified by print
- ✓ GPS dims (obs[5:8]) are 0.0 without route_cache; non-zero with route_cache loaded
- ✓ Baseline rule total fuel recorded in grams (reference for Day 3)
- ✓ PPO training starts and TensorBoard ep_rew_mean rising by step 100k
- ✓ models/best_model.zip exists after EvalCallback fires at 50k steps

DAY 3 · WED
Evaluation, ONNX Export & Dashboard
SIL validation · ONNX · Streamlit · Final report figures

Part 3A · Software-in-the-Loop Evaluation (eval/sil_evaluate.py)

- Load models/best_model.zip. Run deterministic=True evaluation on WLTC, FTP-75, US06. Run each ≥ 3 times and average.
- Record per cycle: total fuel (g), final SOC, SOC RMSE from 60 %, episode return, mode counts per segment, SPORT usage %.
- Compare RL fuel vs rule-based baseline. Compute % fuel savings. Gate: WLTC savings must exceed 5 %.
- Generate publication-quality plots (300 dpi): SOC trajectory, cumulative fuel, mode histogram by segment, reward curve.
- Generate fuel comparison bar chart: 3 groups (WLTC, FTP-75, US06), 4 bars (Random, Rule-based, RL PPO, RL+GPS). Save to eval/figures/.

Part 3B · ONNX Export (training/export_onnx.py)

- Load models/best_model.zip via PPO.load(). Access underlying policy network.
- Create dummy input shape (1, 8) — matches 8-dimensional GPS-enriched observation.
- Export with torch.onnx.export: opset_version=17, input_names=['obs'], output_names=['action_logits']. Save to models/ths_policy.onnx.
- Verify file exists (~200–250 KB — slightly larger than v1.0 due to 8-dim input). Validate ONNX Runtime inference vs SB3 prediction for same dummy input.
- Benchmark: 1000 sequential sess.run() calls on PC. Target < 2 ms per call.
- If RPi available: copy ths_policy.onnx via scp. Run same benchmark. Target < 5 ms per call.

Part 3C · Streamlit Dashboard (app/streamlit_dashboard.py)

- Sidebar: drive cycle selector (WLTC / FTP-75 / US06), agent mode (RL PPO / Rule-based / Random), optional GPS route JSON upload.
- 'Run Episode' button: executes one deterministic episode, stores per-step telemetry in st.session_state.
- SOC trajectory chart: line chart SOC % vs timestep, horizontal reference at 60 %, colour bands for ECO/NORMAL/PWR/EV/SPORT selections.
- Cumulative fuel chart: 4 lines — RL, RL+GPS, rule-based, random baselines.
- Mode distribution donut chart: 5 slices (ECO/NORMAL/PWR/EV/SPORT) with % labels.
- GPS map panel (Folium): if route JSON uploaded, show route polyline coloured by recommended mode per segment.
- Summary metrics table: total fuel (g), fuel savings (%), SOC RMSE, SPORT usage %, episode duration.
- Live replay mode: re-reads ths2_kpis.csv row by row at 10 Hz using st.empty().

Part 3D · Git Release & Documentation

- Write comprehensive README.md: project overview, installation, Day 1–4 run instructions, output file descriptions, dashboard guide.
- Add architecture diagram (ASCII or image): drive cycle CSV / GPS route → THSIIController → Gymnasium → PPO → ONNX → Streamlit.
- Commit all code, models/, eval/figures/, gps/cache/, requirements.txt. Tag release as v2.0-sil.

Day 3 — Checkpoints (Core Project Complete)

- ✓ RL fuel savings on WLTC > 5 % vs rule-based baseline — confirmed in SIL evaluation
- ✓ SOC RMSE < ±5 % of 60 % across all three drive cycles
- ✓ models/ths_policy.onnx exists, ~200–250 KB, ONNX Runtime inference validated on 8-dim obs
- ✓ ONNX inference latency < 2 ms per call on PC
- ✓ eval/figures/ contains SOC trace, fuel bar chart (4 bars), mode histogram — all at 300 dpi
- ✓ Streamlit dashboard launches and GPS map panel renders when route JSON provided
- ✓ Git tag v2.0-sil committed with all deliverables

DAY 4 · THU [NEW]
GPS Route Integration & Predictive EMS
Route segmentation · Elevation · Traffic · Pre-trip mode planning

Scope
Day 4 integrates the GPS pipeline built in Chapter 6 with the trained PPO agent for a full end-to-end predictive EMS demo. The RL policy from Day 3 is used as-is — no retraining required unless GPS features were not included in training (see note below).

Part 4A · GPS Route Fetcher (gps/route_fetcher.py)

- Register for a free OpenRouteService (ORS) API key at openrouteservice.org. Alternatively, run a local OSRM instance for fully offline operation.
- Implement fetch_route(origin_ll, destination_ll, api_key): call ORS directions endpoint (profile: driving-car). Parse GeoJSON response to extract waypoints (lat/lon) and road class tags.
- Implement get_elevation_profile(waypoints): call Open-Elevation API (https://api.open-elevation.com/api/v1/lookup) in batches of 100 points. Return elevation list aligned with waypoints.
- Implement compute_grades(waypoints, elevations): apply Savitzky-Golay filter (scipy.signal.savgol_filter, window=11, polyorder=2) to smooth elevation before computing grade_rad at each waypoint.
- Implement classify_segments(waypoints, road_classes, grades): group consecutive waypoints of the same road class into RouteSegment objects. Override segment_type based on grade thresholds (grade > 4 % → uphill flag, grade < -2 % → downhill flag).
- Save output to gps/cache/<route_hash>.json where route_hash is the MD5 of 'origin_lat,origin_lon,dest_lat,dest_lon'. This prevents redundant API calls on re-runs.

Part 4B · Traffic Data Integration (gps/traffic.py)

- If using ORS: compare duration (current) vs. duration_typical (free-flow) per segment. Compute congestion_ratio = duration / duration_typical. Map to traffic_density: ratio 1.0 → 0.0, ratio ≥ 2.5 → 1.0.
- If using HERE Traffic API: fetch flow tiles for the route bounding box. Sample speed and freeflow_speed per waypoint. Compute density = 1 - (speed / freeflow_speed).
- Store traffic_density per RouteSegment in the route cache JSON.
- For offline testing with drive cycles (no real route), set traffic_density=0.0 for highway segments and 0.5 for urban segments as a default.

Part 4C · GPS-Enriched Training Episodes (gps/gps_env_wrapper.py)

If the model trained on Day 2 did NOT include GPS features in training (obs[5:8] were always 0.0), it must be retrained with GPS features injected to achieve GPS-aware mode selection. Use the following approach:

- Create 5 representative synthetic routes covering: flat urban, hilly rural, steep highway climb, mixed (urban → rural → highway), and downhill suburban. Store each as a route cache JSON with synthetic waypoints.

- Create GPSWrappedEnv that randomly selects one of the 5 routes each episode reset(), matching the drive cycle to the route type (US06 → highway route, FTP75 → urban, WLTC → mixed).
- Retrain from the Day 2 best checkpoint: PPO.load('models/best_model.zip') → model.set_env(GPSWrappedEnv()) → model.learn(total_timesteps=300_000). This fine-tunes rather than training from scratch.
- Save GPS-fine-tuned model as models/ths_agent_gps.zip. Export a new ONNX: models/ths_policy_gps.onnx.

Part 4D · Pre-Trip Mode Sequence Generator (gps/pre_trip_planner.py)

The pre-trip planner uses the trained GPS-aware agent to recommend a mode sequence before departure — matching the academic report's supervisory-layer concept.

- Load models/ths_agent_gps.zip. Load route cache for the planned trip.
- Simulate the route: for each RouteSegment, construct a mock observation using segment features (segment mid-speed, assumed SOC=0.60, segment grade, segment_type, acceleration=0, lookahead grade, traffic density, dist_to_next).
- Call model.predict(obs, deterministic=True) to get the recommended mode for each segment.
- Output: a list of (segment_id, segment_start_km, segment_end_km, recommended_mode, grade_rad, traffic_density). Save to gps/pre_trip_plan.json and print a human-readable summary.
- This plan is advisory — the online RL agent running during the trip may override it based on real-time conditions.

Part 4E · GPS Validation Run

- Run a full WLTC episode with the GPS-fine-tuned model and the mixed synthetic route loaded. Compare fuel and SOC against the no-GPS baseline from Day 3.
- Verify GPS anticipation rate: at grade transitions (grade > 3 %), what % of the time did the agent select PWR or SPORT at least 1 segment before the grade event? Target > 60 %.
- Generate a GPS validation plot: SOC trajectory, mode selections, and grade profile on the same time axis. Save to eval/figures/gps_validation.png at 300 dpi.
- Add the GPS map route to the Streamlit dashboard: load gps/pre_trip_plan.json and colour-code the Folium polyline by recommended mode.

Day 4 — Checkpoints (GPS Integration Complete)

- ✓ gps/route_fetcher.py fetches ORS route and elevation — route_cache JSON saved
- ✓ Gradients computed and smoothed — grade_rad values plotted and visually correct
- ✓ Traffic density values present in route cache JSON for all segments
- ✓ THSEnv with route_cache loaded — obs[5:8] non-zero and change at segment boundaries
- ✓ GPS-fine-tuned model ths_agent_gps.zip saved and produces valid predictions
- ✓ Pre-trip plan JSON generated with mode recommendation per segment
- ✓ GPS anticipation rate > 60 % on validation run
- ✓ Streamlit GPS map panel renders route with mode colour-coding

DAY 5 · FRI [OPTIONAL]

CARLA Live Co-Simulation Bonus

Optional bonus — requires CARLA 0.9.x installed on host

Optional — Prerequisites

- CARLA 0.9.x installed and CarlaUE4.sh server launchable on port 2000
- Day 3 RL policy (models/best_model.zip) already trained and validated
- Day 4 GPS pipeline optional but recommended — CARLA provides its own road topology
- The RL policy does NOT need retraining for CARLA integration

Part 5A · Understanding CarlaTHSIISimulation in modeling1.py

- Read the CarlaTHSIISimulation class carefully. Identify _select_drive_mode(speed, soc, grade, seg) — the only method you will override.
- Note how the base class calls _select_drive_mode() on each tick and uses the returned DriveMode to govern the physics engine.
- Confirm that importing modeling1.py with CARLA unavailable raises ImportError — guard your bridge script with try/except.

Part 5B · Building the RL Bridge (carla/carla_rl_bridge.py)

- Create RLBridgeSimulation subclass inheriting from CarlaTHSIISimulation.
- In __init__, call super().__init__() then load PPO.load('models/ths_agent_gps.zip'). Store as self.policy.
- Override _select_drive_mode(speed, soc, grade, seg): build 8-element obs (speed/30, soc/100, grade/0.3, seg/2, 0.0, gps_lookahead/0.3, traffic/1.0, dist/1000). Call self.policy.predict(obs, deterministic=True). Return DriveMode enum for the integer action.
- If CARLA road topology is available, extract OSM road class from the current CARLA waypoint and inject as seg. This replaces the drive-cycle segment_type.
- Log each tick: timestep, DriveMode, SOC, speed, CARLA location. Save to carla/carla_rl_log.csv.

Part 5C · Running and Validating

- Start CarlaUE4.sh → python carla/carla_rl_bridge.py --map Town03. Vehicle should spawn and move.
- Monitor SOC and mode in terminal. Compare mode distribution histogram vs SIL results from Day 3.
- Qualitative consistency is the goal — CARLA physics variability means exact fuel match is not expected.

Day 5 — Optional Checkpoints

- ✓ [Optional] CARLA server connects without error, vehicle spawns in Town03
- ✓ [Optional] _select_drive_mode() called each tick — SPORT action (4) reachable
- ✓ [Optional] carla_rl_log.csv populated with mode, SOC, speed, location columns

- ✓ [Optional] Mode distribution qualitatively consistent with SIL results
- ✓ [Optional] No Python exceptions during a full 5-minute CARLA run

DAY 6 · SAT [OPTIONAL]

CAN Bus Hardware-in-the-Loop

Optional HIL — requires RPi 4 + MCP2515 CAN HAT

Optional — Prerequisites

- Raspberry Pi 4 (4 GB RAM) + MCP2515 CAN HAT + USB-CAN adapter for PC
- models/ths_policy_gps.onnx from Day 4 (or ths_policy.onnx from Day 3)
- ONNX model accepts 8-dimensional input (verify before copying to RPi)

Part 6A · Hardware Assembly & OS Configuration

- Mount MCP2515 CAN HAT on RPi 4. Connect PC USB-CAN adapter via twisted-pair to HAT. Place 120 Ω terminators at both ends.
- Flash Raspberry Pi OS 64-bit (Bookworm). Enable SPI: raspi-config → Interface Options → SPI.
- Add overlay: dtoverlay=mcp2515-can0,oscillator=8000000,interrupt=25 to /boot/firmware/config.txt. Reboot. Verify can0 in 'ip link show'.
- Bring up CAN on both sides: sudo ip link set can0 type can bitrate 500000 && sudo ip link set can0 up.
- Loopback test: cansend on PC → candump on RPi. Confirm correct frame content.

Part 6B · CAN Protocol — Frame Definitions

Frame 0x100 — Vehicle State (PC → RPi) · DLC: 8 bytes · 100 ms cycle

Bytes	Signal	Type	Encoding / Notes
0–1	velocity	uint16	v × 100 [cm/s] — 0.01 m/s resolution
2	SOC	uint8	SOC × 100 [%] — 1 % resolution
3	road_grade	int8	grade × 50 [°] — 0.02° resolution
4	segment_type	uint8	0=urban / 1=rural / 2=highway
5	acceleration	int8	a × 10 [dm/s²] — 0.1 m/s² resolution
6	frame_counter	uint8	Rolling 0–255
7	checksum	uint8	XOR bytes 0–6

GPS Dims in CAN v2.0

The 8-dim GPS observation (gps_lookahead_grade, gps_traffic_density, gps_dist_to_next_seg) is NOT transmitted via CAN — it is pre-loaded from gps/cache/ on the RPi before the HIL run starts and indexed by segment_type. This avoids expanding the CAN frame size.

Frame 0x200 — Mode Decision (RPi → PC) · DLC: 4 bytes

Bytes	Signal	Type	Encoding / Notes
0	mode_action	uint8	0=ECO / 1=NORMAL / 2=PWR / 3=EV / 4=SPORT
1	latency_ms	uint8	ONNX inference time in ms
2	frame_counter	uint8	Echo of received 0x100 counter
3	checksum	uint8	XOR bytes 0–2

Part 6C · PC-Side CAN Driver (hil/pc_side_can.py)

- encode_state_frame(): pack 5 values into 8-byte 0x100 frame via struct.pack. Apply XOR checksum over bytes 0–6.
- send_state(): take THSEnv obs array (first 5 dims only for CAN — GPS dims handled by RPi) and frame_counter, encode, transmit via bus.send() on can0.
- receive_mode(): bus.recv(timeout=0.05) for 0x200 frame. Decode action byte and latency. Validate checksum. On timeout, return previous mode (fail-safe hold).
- HIL loop: for each WLTC timestep — send state, receive mode, apply DriveMode to Gymnasium, advance step, log to hil/hil_log.csv.

Part 6D · RPi-Side ONNX Controller (hil/rpi_controller.py)

- Copy ths_policy_gps.onnx to ~/models/ on RPi. Install ONNX Runtime: pip install onnxruntime.
- Pre-load GPS route cache for the WLTC route at startup. Index by segment_type (urban/rural/highway) to get lookahead_grade, traffic_density, dist_to_next estimates.
- Receive loop: block on bus.recv() for 0x100 frames. Decode 8-byte payload. Reconstruct 5-dim obs from CAN. Look up GPS dims from pre-loaded cache by segment_type. Build full 8-dim obs array.
- ONNX inference: sess.run(None, {input_name: obs}). Argmax of logits → action 0–4. Measure latency with time.perf_counter().
- Encode 0x200 mode frame with 5-mode action byte. Send immediately after inference.
- Watchdog: no 0x100 frame within 1 s → hold last mode, print warning.

Part 6E · HIL Validation (eval/hil_evaluate.py)

- Start rpi_controller.py on RPi first, then pc_side_can.py on PC.
- Run full WLTC (1800 steps × 100 ms = 180 s real time). Monitor for CAN errors.
- After run: load hil_log.csv. Compute mean and max latency. Verify mean < 5 ms, max < 10 ms.
- Compare HIL fuel, SOC, mode distribution against Day 3 SIL results.
- CAN error rate: (dropped + corrupt) / total × 100. Target < 0.1 %.
- Monitor RPi with htop: peak RAM < 200 MB, average CPU < 30 %.

Parameter	Value / Note
Frame period (0x100)	100 ms — 10 Hz control loop
Max inference latency	< 10 ms on RPi 4 CPU
CAN frame timeout	50 ms — bus.recv(timeout=0.05)

Parameter	Value / Note
Missing frame action	Hold previous — no mode switching on drop
Bus-off recovery	Automatic MCP2515 hardware recovery
Error frame target	< 0.1 % — monitor via ip -s link can0

Day 6 — Optional Checkpoints

- ✓ [Optional] CAN loopback test passes — cansend on PC received cleanly by candump on RPi
- ✓ [Optional] ths_policy_gps.onnx benchmark on RPi < 5 ms per call (8-dim input)
- ✓ [Optional] Full WLTC HIL episode completes 1800 steps without CAN error
- ✓ [Optional] Mode_action byte correctly encodes SPORT as value 4 in 0x200 frame
- ✓ [Optional] Mean HIL latency < 5 ms, max < 10 ms — confirmed in hil_log.csv
- ✓ [Optional] CAN frame error rate < 0.1 % over full episode
- ✓ [Optional] RPi peak RAM < 200 MB, CPU < 30 % — confirmed via htop
- ✓ [Optional] HIL fuel and SOC consistent with Day 3 SIL results

13 · Project Deliverables

Software Deliverables

ID	Path	Description	Status
SW-01	env/ths_env.py	Custom Gymnasium env (8-dim obs, 5 actions)	Mandatory
SW-02	modeling1.py	THS-II v4 physics engine — provided, DO NOT MODIFY	Mandatory
SW-03	env/drive_cycles/*.csv	WLTC, FTP-75, US06 drive cycle data	Mandatory
SW-04	training/train_ppo.py	PPO training script with EvalCallback	Mandatory
SW-05	training/baseline_rule.py	Rule-based EMS comparator (5-mode)	Mandatory
SW-06	training/export_onnx.py	PyTorch policy → ONNX (8-dim input)	Mandatory
SW-07	eval/sil_evaluate.py	SIL validation on all 3 cycles + plots	Mandatory
SW-08	app/streamlit_dashboard.py	Dashboard with GPS map panel	Mandatory
SW-09	models/best_model.zip	Best SB3 PPO checkpoint (no-GPS)	Mandatory
SW-10	models/ths_policy.onnx	ONNX model — no-GPS (Day 3)	Mandatory
SW-11	gps/route_fetcher.py	ORS route + elevation fetcher	Day 4
SW-12	gps/traffic.py	Traffic density integration	Day 4
SW-13	gps/pre_trip_planner.py	Pre-trip mode sequence generator	Day 4
SW-14	models/ths_agent_gps.zip	GPS fine-tuned SB3 checkpoint	Day 4
SW-15	models/ths_policy_gps.onnx	GPS-aware ONNX model (8-dim)	Day 4
SW-16	carla/carla_rl_bridge.py	CARLA ↔ RL bridge (5-mode)	Optional D5
SW-17	hil/pc_side_can.py	PC CAN state transmitter (5-mode 0x200)	Optional D6
SW-18	hil/rpi_controller.py	RPi ONNX inference + CAN responder	Optional D6
SW-19	eval/hil_evaluate.py	HIL validation with latency logging	Optional D6

Hardware Deliverables (Optional Day 6)

Parameter	Value
HW-01 — Raspberry Pi 4 (4 GB)	HIL target running ONNX inference with 8-dim GPS-aware policy
HW-02 — MCP2515 CAN HAT	SPI-to-CAN interface for Raspberry Pi
HW-03 — USB-CAN Adapter	PC-side CAN interface (Canable / PCAN-USB)

Parameter	Value
HW-04 — Twisted-pair cable	Bus wiring with 120 Ω termination at both ends

14 · Conclusion & Future Work

14.1 Summary

This enhanced v2.0 guide delivers a complete, reproducible pipeline for learning-based Energy Management in a Toyota THS-II hybrid vehicle. The core contribution is a PPO agent trained on standardised drive cycles using the high-fidelity modeling1.py physics engine with a full 5-mode DriveMode action space (ECO / NORMAL / PWR / EV / SPORT). The v2.0 additions include a GPS route planning pipeline (Day 4) that enriches the agent's state with road topology, elevation gradients, and traffic density — enabling predictive, route-aware mode selection before each segment.

14.2 Expected Results

Parameter	Value
Fuel savings (SIL, no GPS)	5–15 % vs. rule-based baseline on WLTC cycle
Fuel savings (GPS-enriched)	Additional 2–5 % from predictive mode anticipation
SOC deviation	< ±5 % RMSE from 60 % across all test cycles
GPS anticipation rate	> 60 % of grade transitions pre-selected correctly
ONNX latency (PC)	< 2 ms per inference call (8-dim input)
ONNX latency (RPi 4)	< 5 ms per inference call [Optional D6]
CAN frame error rate	< 0.1 % over 1800-step WLTC HIL episode
Dashboard	10 Hz replay + GPS Folium map with mode colour-coding
CARLA integration	Qualitative 5-mode RL control in live CARLA traffic [Optional D5]

14.3 Future Work

ID	Topic	Description	Effort
FW-01	Full CARLA Emergent Traffic	Replace prescribed cycles with dynamic CARLA scenarios + GPS integration	High
FW-02	Multi-Objective Reward	Add comfort (jerk) and emissions (NOx) terms to reward function	Medium
FW-03	Real-Time Google Maps	Live Maps API → segment type prediction before trip start	Medium
FW-04	THS-V Migration	Port modeling1.py physics to THS-V powertrain parameters	High
FW-05	dSPACE HIL Upgrade	Automotive-grade HIL certification platform upgrade	High
FW-06	SAC / TD3 Comparison	Benchmark off-policy algorithms for sample efficiency	Low

ID	Topic	Description	Effort
FW-07	Online Adaptation	Fine-tune policy on-device after deployment	High
FW-08	Federated Fleet Learning	Federated RL across multiple vehicles — shared policy backbone	Research
FW-09	Hierarchical RL	High-level GPS planner + low-level torque controller	Research

THS-II EMS RL Pipeline v2.0 · Enhanced Sprint Guide
Oussama Chouaibi · EABA | Znazen Khalil · Karoui Wassim · Chouaibi Mohamed Aziz | INSAT 2025/2026